TECNOLOGO EN ANALISIS Y DESARROLLO DE SOFTWARE

SERVICIO NACIONAL DE APRENDIZAJE CENTRO DE LA INDUSTRIA, LA EMPRESA Y LOS SERVICIOS

MANUAL DE INSTALACIÓN Y CONFIGURACIÓN

PROYECTO: DRIVIQUE

INTEGRANTES:

LAURA VANESSA PEREZ PERDOMO
DANNA VALENTINA BARRIOS PENAGOS
EMILY SHARITH AMEZQUITA SAAVEDRA

FICHA:3145555

2026

Tabla de contenido

1. Introducción y Requisitos del Sistema 4
1.1 Requerimientos de Hardware 4
1.2 Requerimientos de Software y Herramientas Base 4
2. Obtención y Estructura del Ecosistema de Repositorios 5
3. Aprovisionamiento y Configuración de la Base de Datos 6
3.1 Inicialización del Contenedor PostgreSQL Local 6
3.2 Aplicación de Migraciones con Liquibase 6
4. Gestión de Variables de Entorno y Secretos 7
4.1 Archivo .env del Backend (Back-end/.env) 7
4.2 Archivo .env del Frontend Web (front-end_web/web-drivique/.env) 7
5. Compilación y Ejecución del Backend API (Spring Boot) 8
5.1 Compilación y Descarga de Dependencias con Maven 8
5.2 Lanzamiento del Servidor en Perfil de Desarrollo 8
5.3 Verificación de Disponibilidad del Backend 8
6. Instalación y Ejecución del Frontend Web (React 19 + Vite) 9
6.1 Instalación de Dependencias Node.js 9
6.2 Lanzamiento del Servidor de Desarrollo 9
6.3 Verificación Funcional Web 9
7. Aplicación Móvil: Ejecución Local y Guía Detallada del APK Android 10
7.1 Instalación de Dependencias Móviles 10
7.2 Ejecución Rápida en Desarrollo con Expo Go 10
7.3 Generación del Paquete Instalador APK (EAS Build) 10
7.4 Método A: Instalación Directa en el Celular (Sideloading) 11
7.5 Método B: Instalación Automatizada mediante ADB (Android Debug Bridge) 11
7.6 Buenas Prácticas y Solución de Fallos en APK 12
8. Despliegue Orquestado en Servidor de Producción (Hostinger VPS) 13
8.1 Estructura en el Servidor (/opt/drivique/) 13
8.2 Ejecución del Script de Despliegue Automatizado 13
9. Checklist de Verificación y Pruebas de Humo (Smoke Tests) 14
10. Guía de Solución de Problemas Frecuentes (Troubleshooting) 15

10.1 Conflicto de Puertos en Base de Datos (Port Already Allocated)..... 15
10.2 Error de CORS en el Navegador Web..... 15
10.3 La App Móvil o APK no Conecta con la API Local..... 15
10.4 Fallo de Validación Hibernate (SchemaManagementException).....15

1. Introducción y Requisitos del Sistema

El presente Manual de Instalación y Configuración establece la guía técnica estandarizada, paso a paso y validada para aprovisionar, compilar y ejecutar de forma exitosa la plataforma Drivique tanto en un entorno local de desarrollo como en infraestructura de producción y dispositivos móviles Android.

Drivique es una plataforma integral de gestión de flotas y reservas de vehículos compuesta por cuatro módulos desacoplados: un motor de base de datos relacional PostgreSQL con migraciones versionadas por Liquibase, una API REST robusta en Spring Boot (Java 21 LTS), una aplicación web Single Page Application (React 19 + Vite) y una aplicación móvil nativa con empaquetado APK para Android (React Native + Expo SDK 57 con soporte EAS Build).

1.1 Requerimientos de Hardware

Componente de Hardware	Requisito Mínimo (Local)	Recomendado (Desarrollo)	Servidor Producción (VPS)
Procesador (CPU)	2 Núcleos (x86_64 / ARM64)	4 Núcleos o superior	4 vCPU dedicadas
Memoria RAM	8 GB RAM	16 GB RAM	8 GB RAM + 2 GB Swap
Almacenamiento Libre	10 GB libres en SSD	20 GB libres en SSD NVMe	50 GB SSD NVMe
Conectividad	Conexión a Internet (10 Mbps)	Banda Ancha (50 Mbps)	100 Mbps simétricos (Hostinger)

1.2 Requerimientos de Software y Herramientas Base

Herramienta / Entorno	Versión Obligatoria	Uso Principal en Drivique	Verificación en Terminal
Sistema Operativo	Windows 11 / Ubuntu 22.04+ / macOS 13+	Plataforma anfitriona de desarrollo	winver o uname -a
Git SCM	Git 2.40.0 o superior	Control de versiones y clonación de repositorios	git --version
Docker Engine & Compose	Docker 24.x+ / Compose v2.20+	Contenedores de PostgreSQL y servicios	docker --version && docker compose version
Java Development Kit	JDK 21 LTS (Eclipse Temurin / OpenJDK)	Compilación y ejecución del Backend Spring Boot	java -version
Node.js & npm	Node.js v22.x LTS / npm 10.x+	Compilación de Frontend Web y Frontend Móvil	node -v && npm -v
Expo CLI & EAS CLI	Expo SDK ~57.0.21 / EAS CLI >= 16.0.0	Desarrollo y compilación de instalables APK	npx expo --version && npx eas --version
Android Platform Tools	ADB v34+ (opcional)	Instalación y depuración directa por USB	adb version

2. Obtención y Estructura del Ecosistema de Repositorios

El código fuente de Drivique está distribuido en repositorios oficiales bajo la organización de GitHub project-drivique. Para mantener la coherencia de rutas relativas y facilitar el despliegue orquestado, se debe organizar una carpeta raíz de trabajo:

```
# 1. Crear la carpeta de trabajo principal
mkdir drivique
cd drivique


# 2. Clonar el repositorio de Base de Datos y Changelogs
git clone https://github.com/project-drivique/database.git


# 3. Clonar el repositorio del Backend API (Spring Boot)
git clone https://github.com/project-drivique/Back-end.git


# 4. Clonar el repositorio del Frontend Web (React 19)
git clone https://github.com/project-drivique/front-end_web.git


# 5. Clonar el repositorio del Frontend Móvil (React Native / Expo)
git clone https://github.com/project-drivique/front-end_movil.git


# 6. Clonar el repositorio de Documentación institucional
git clone https://github.com/project-drivique/Documentation.git
```

Estructura final del directorio de trabajo:

```
drivique/
├── Back-end/                # Servidor de API REST Java 21 / Spring Boot 4.0.8
├── database/                # Docker local, scripts initdb y changelogs Liquibase
├── deploy/                  # Orquestación de producción (docker-compose, Caddy, scripts)
├── front-end_movil/         # App móvil híbrida (Expo SDK 57 / React Native / Android)
│   └── app-drivique/
├── front-end_web/           # Portal web comercial y administrativo (React 19 / Vite)
│   └── web-drivique/
└── Documentation/           # Informes técnicos, runbooks y manuales del proyecto
```

3. Aprovisionamiento y Configuración de la Base de Datos

Drivique utiliza PostgreSQL 17 Alpine con extensiones criptográficas (pgcrypto) y segregación de 10 esquemas temáticos (iam, core, location, fleet, catalog, rental, contract, billing, support, audit). Para evitar instalaciones manuales invasivas en el sistema operativo, la base de datos se levanta en un contenedor Docker aislado.

3.1 Inicialización del Contenedor PostgreSQL Local

1. Navega al subdirectorio docker del repositorio database:

```
cd database/docker
docker compose up -d
```

2. Verificación de salud del contenedor:

```
docker ps --filter "name=postgres"
```

El contenedor inicializa automáticamente los esquemas y roles de base de datos definidos en database/docker/initdb/01-create-application-role.sh. En el entorno local de desarrollo, el puerto expuesto en el host es el 5433 (o 5432) mapeado hacia el puerto interno 5432 del contenedor.

3.2 Aplicación de Migraciones con Liquibase

Liquibase gestiona de forma automática y versionada el ciclo de vida DDL/DML. Al iniciar el Backend, Liquibase procesa el archivo maestro db.changelog-master.yaml y ejecuta de forma secuencial los 41 releases sin riesgo de corrupción ni borrado de datos:

- **Changelogs Base (001-enable-pgcrypto.sql):** Habilita la extensión pgcrypto para generación de UUIDs y encriptación nativa.
- **Definición DDL (01_ddl/changelog.yaml):** Generación estructural de los 10 esquemas y sus llaves primarias/foráneas.
- **Datos Iniciales DML (02_dml/changelog.yaml):** Carga de catálogos semilla (marcas, categorías, sucursales y roles del sistema).
- **Releases Funcionales (hu-bd-03.yaml a hu-bd-41.yaml):** Migraciones incrementales de casos de uso y remediaciones.

4. Gestión de Variables de Entorno y Secretos

Cada capa de la plataforma se parametriza mediante variables de entorno para evitar almacenar credenciales sensibles en el control de versiones.

4.1 Archivo .env del Backend (Back-end/.env)

Crea o edita el archivo .env en la raíz del repositorio Back-end/ basándote en la siguiente configuración estándar de desarrollo local:

```
# --- BASE DE DATOS LOCAL ---
DB_URL=jdbc:postgresql://localhost:5433/drivique_dev
DB_USERNAME=drivique_app
DB_PASSWORD=change-me-app-password

# --- SEGURIDAD Y JWT ---
JWT_SECRET=404E635266556A586E3272357538782F413F4428472B4B6250645367566B5970
JWT_EXPIRATION_MS=86400000
CORS_ALLOWED_ORIGINS=http://localhost:5173,http://localhost:3000,http://localhost:8081

# --- PASARELA DE PAGOS WOMPI (SANDBOX) ---
WOMPI_PUBLIC_KEY=pub_test_Q5yDA9xoKdePzhSGeVe9KStAO6OfCvyP
WOMPI_PRIVATE_KEY=prv_test_Xz8Y9wK1...
WOMPI_EVENT_SECRET=test_events_uK90...

# --- SERVICIOS EXTERNOS ---
FRANKFURTER_TIMEOUT_MS=5000
SPRING_PROFILES_ACTIVE=dev
```

4.2 Archivo .env del Frontend Web (front-end_web/web-drivique/.env)

```
VITE_API_BASE_URL=http://localhost:8080/api/v1
VITE_WOMPI_PUBLIC_KEY=pub_test_Q5yDA9xoKdePzhSGeVe9KStAO6OfCvyP
VITE_DEFAULT_LANGUAGE=es
```

5. Compilación y Ejecución del Backend API (Spring Boot)

El servidor Backend centraliza la lógica de negocio, validaciones transaccionales y seguridad. Se ejecuta sobre Java 21 LTS.

5.1 Compilación y Descarga de Dependencias con Maven

Abre una terminal en el directorio Back-end y ejecuta el wrapper de Maven integrado:

```
cd Back-end

# En Windows:
.\mvnw.cmd clean install -DskipTests

# En Linux / macOS:
./mvnw clean install -DskipTests
```

5.2 Lanzamiento del Servidor en Perfil de Desarrollo

```
# Ejecutar con Spring Boot Maven Plugin
./mvnw spring-boot:run -Dspring-boot.run.profiles=dev
```

5.3 Verificación de Disponibilidad del Backend

Una vez que la consola imprima el mensaje Started DriviqueApiApplication in X.XXX seconds, verifica los siguientes puntos de acceso en tu navegador:

- **Swagger OpenAPI:** Documentación interactiva de la API: http://localhost:8080/api/swagger-ui.html
- **Spring Actuator:** Monitoreo de salud del servicio: http://localhost:8080/api/actuator/health (debe responder {"status":"UP"})
- **Catálogo REST:** Búsqueda básica de flota: http://localhost:8080/api/v1/vehicles/search?size=1 (retorna código 200 OK)

6. Instalación y Ejecución del Frontend Web (React 19 + Vite)

El Frontend Web es una SPA moderna desarrollada con React 19, Tailwind CSS y Vite que sirve como portal público de reservas y consola de administración.

6.1 Instalación de Dependencias Node.js

Abre una nueva terminal en el subdirectorio web-drivique:

```
cd front-end_web/web-drivique
npm install
```

6.2 Lanzamiento del Servidor de Desarrollo

```
npm run dev
```

El compilador Vite iniciará el servidor de desarrollo local en http://localhost:5173.

6.3 Verificación Funcional Web

Abre el navegador en http://localhost:5173. Deberás observar la página de bienvenida, el selector de ciudades, las fechas de reserva y el listado de vehículos disponibles consumiendo directamente la API local.

7. Aplicación Móvil: Ejecución Local y Guía Detallada del APK Android

La aplicación móvil de Drivique está desarrollada sobre React Native 0.86 y Expo SDK 57 (paquete com.drivique.app), permitiendo dos modalidades de uso: ejecución en tiempo real mediante el motor de desarrollo Expo Go, o empaquetado e instalación directa como binario APK independiente para dispositivos físicos Android.

7.1 Instalación de Dependencias Móviles

Abre una terminal en el subdirectorio app-drivique e instala las dependencias del ecosistema Expo:

```
cd front-end_movil/app-drivique
npm install
```

7.2 Ejecución Rápida en Desarrollo con Expo Go

1. Configuración de Red: Cuando pruebas en un smartphone físico a través de Expo Go, el dispositivo no puede usar localhost porque buscaría en sí mismo. Debes consultar la IP de tu PC en la red local mediante ipconfig en Windows (ej. 192.168.1.15) y configurarla en la variable de entorno o en la URL base del servicio de API:

```
// src/services/http/apiClient.ts o .env
EXPO_PUBLIC_API_URL=http://192.168.1.15:8080/api/v1
```

2. Iniciar el servidor Metro Bundler:

```
npx expo start
```

3. Conexión:

- **Dispositivo Físico:** Abre la cámara (iOS) o la app Expo Go (Android) y escanea el código QR proyectado en la terminal.
- **Emulador Android Studio:** Pulsa la tecla 'a' en la consola para compilar y desplegar automáticamente en el AVD activo.
- **Entorno Web:** Pulsa la tecla 'w' para previsualizar la interfaz en el navegador web.

7.3 Generación del Paquete Instalador APK (EAS Build)

El proyecto Drivique cuenta con el perfil 'preview' configurado en front-end_movil/app-drivique/eas.json con distribution: internal y buildType: apk (package: com.drivique.app). Esto genera un instalador universal (.apk) listo para ser instalado en cualquier celular Android sin depender de la tienda Google Play Store.

Para generar un nuevo APK independiente:

```
# 1. Instalar la CLI de EAS si no está presente
npm install -g eas-cli

# 2. Iniciar sesión en la plataforma Expo / EAS
eas login

# 3. Compilar el APK en la nube con el perfil preview
cd front-end_movil/app-drivique
eas build -p android --profile preview
```

Al culminar la compilación, EAS te proporcionará un enlace directo de descarga para el archivo drivique.apk, así como un código QR en consola para descargarlo de forma directa en el teléfono.

7.4 Método A: Instalación Directa en el Celular (Sideloading)

El sideloading es el método estándar y más amigable para cualquier evaluador o instructor que requiera probar el aplicativo Drivique en su propio teléfono móvil:

- **Paso 1 (Transferencia o Descarga):** Transfiere el archivo drivique.apk a la memoria del teléfono mediante cable USB, o descárgalo directamente abriendo el enlace de EAS en el navegador Google Chrome del smartphone.
- **Paso 2 (Habilitar Orígenes Desconocidos):** Android por seguridad bloquea instalaciones fuera de Google Play. Ingresa a Ajustes > Aplicaciones > Acceso especial a apps > Instalar aplicaciones desconocidas. Selecciona la aplicación que usarás para abrir el instalador (por ejemplo Google Chrome o el Administrador de Archivos) y activa la opción 'Permitir desde esta fuente'.
- **Paso 3 (Ejecución del Instalador):** Abre el Administrador de Archivos del celular, ve a la carpeta Descargas (Downloads) y pulsa sobre drivique.apk. En el cuadro de diálogo emergente que pregunta '¿Deseas instalar esta aplicación?', presiona el botón Instalar.
- **Paso 4 (Permisos y Primer Arranque):** Una vez completada la barra de progreso, pulsa en 'Abrir'. Al arrancar por primera vez, concede los permisos de Ubicación y Notificaciones necesarios para el cálculo de sucursales y confirmación de reservas vehiculares.

7.5 Método B: Instalación Automatizada mediante ADB (Android Debug Bridge)

Este método es ideal para ingenieros de desarrollo, pruebas de humo y entornos de integración continua conectados por cable USB:

1. Requisitos Previos en el Computador y Móvil:

- **Activar USB Debugging:** En el teléfono: Ve a Ajustes > Acerca del teléfono > pulsa 7 veces continuas sobre 'Número de compilación' hasta activar el modo desarrollador. Luego ve a Ajustes > Opciones de desarrollador y activa la casilla 'Depuración por USB'.
- **Instalar ADB:** Descarga Android SDK Platform-Tools oficial de Google y verifica que el binario adb esté disponible en tu consola.

2. Verificación de Dispositivo Conectado:

```
adb devices
# Respuesta esperada:
# List of devices attached
# 19281FDE400129    device
```

3. Comando de Instalación con Concesión Automática de Permisos:

```
# -r = Reinstala conservando datos previos de la app
# -g = Otorga automáticamente todos los permisos en tiempo de instalación
adb install -r -g "C:\ruta\al\archivo\drivique.apk"
```

La terminal responderá con la palabra 'Success', dejando la aplicación lista y visible en el menú principal del teléfono.

7.6 Buenas Prácticas y Solución de Fallos en APK

Incidente / Escenario	Causa Técnica Raíz	Acción de Remediación Obligatoria
INSTALL_FAILED_ALREADY_EXISTS	Existe una versión anterior con firma digital distinta en el dispositivo.	Desinstalar la versión anterior con: adb uninstall com.drivique.app e intentar de nuevo.
Error de Análisis del Paquete	El archivo APK se descargó de forma incompleta o la versión de Android es < 10.	Verificar el tamaño del APK descargado y comprobar que Android sea 10 (API 29) o superior.
Pantalla en blanco al abrir el APK	La URL de la API (EXPO_PUBLIC_API_URL) no es alcanzable desde la red del teléfono.	Comprobar que el teléfono tenga acceso a https://drivique.tech o a la IP del servidor API.

8. Despliegue Orquestado en Servidor de Producción (Hostinger VPS)

En el servidor de producción Hostinger Cloud VPS (Ubuntu 24.04 LTS, IP 2.25.75.54), todo el ecosistema corre contenerizado bajo Docker Compose y protegido por Caddy 2.

8.1 Estructura en el Servidor (/opt/drivique/)

```
/opt/drivique/
├── Back-end/
├── database/
├── front-end_web/
├── front-end_movil/
└── deploy/
    ├── docker-compose.yml
    ├── Caddyfile
    ├── .env
    └── scripts/
        ├── deploy.sh
        ├── backup.sh
        └── restore.sh
```

8.2 Ejecución del Script de Despliegue Automatizado

```
# 1. Conexión SSH al servidor
ssh deploy@2.25.75.54


# 2. Navegar al directorio de despliegue y ejecutar
cd /opt/drivique/deploy
./scripts/deploy.sh
```

El script maestro deploy.sh automatiza todo el ciclo de actualización:

- **1. Git Sync:** Descarga las últimas actualizaciones de la rama main en los repositorios con git pull.
- **2. Backup Preventivo:** Genera un dump lógico comprimido preventivo de la base de datos antes de aplicar cambios.
- **3. Build Docker:** Recompila las imágenes multi-stage de Nginx y Spring Boot con docker compose build.
- **4. Migración DB:** Ejecuta el contenedor puntual de Liquibase para aplicar cambios DDL/DML nuevos.
- **5. Up Contenedores:** Levanta los contenedores en segundo plano sin tiempo de inactividad (Zero-Downtime).
- **6. Healthcheck:** Espera hasta 48 iteraciones hasta certificar que el healthcheck de la API responde healthy.

9. Checklist de Verificación y Pruebas de Humo (Smoke Tests)

Una vez finalizada la instalación local o en producción, ejecuta este checklist para validar la integridad operativa:

N°	Componente Evaluado	Comando o URL de Verificación	Resultado Esperado
1	Motor PostgreSQL	docker ps --filter 'name=postgres'	Contenedor en estado 'Up' (saludable)
2	Esquemas Liquibase	docker logs <api-container> | grep Liquibase	'Liquibase: Update has been successful'
3	Salud del Backend	curl -s http://localhost:8080/api/actuator/health	{"status":"UP"}
4	Swagger OpenAPI	Navegador en http://localhost:8080/api/swagger-ui.html	Carga completa de la documentación interactiva
5	Frontend Web	Navegador en http://localhost:5173	Renderizado del catálogo y barra de navegación
6	Frontend Móvil (APK)	Ejecución en celular o adb shell monkey	Carga del layout móvil y login sin cierres inesperados
7	Conexión End-to-End	Registro de usuario de prueba desde la UI Web	Código HTTP 201 Created y token JWT emitido

10. Guía de Solución de Problemas Frecuentes (Troubleshooting)

A continuación se recopilan los incidentes técnicos más comunes durante la instalación y sus soluciones concretas:

10.1 Conflicto de Puertos en Base de Datos (Port Already Allocated)

- **Síntoma:** Docker arroja error bind: address already in use indicando que el puerto 5432 o 5433 ya está ocupado por una instancia local de PostgreSQL instalada en tu equipo.
- **Solución:** Detén el servicio de PostgreSQL local de tu máquina (en Windows: services.msc -> detener 'postgresql-x64-XX'; en Linux: sudo systemctl stop postgresql) o cambia el mapeo del host en database/docker/docker-compose.yml a un puerto libre como 5434:5432.

10.2 Error de CORS en el Navegador Web

- **Síntoma:** En la consola de desarrollador del navegador aparece Access to XMLHttpRequest at ... has been blocked by CORS policy.
- **Solución:** Verifica que el puerto exacto donde está corriendo Vite (ej. http://localhost:5173) esté incluido en la variable CORS_ALLOWED_ORIGINS de tu archivo Back-end/.env y reinicia el servidor Spring Boot.

10.3 La App Móvil o APK no Conecta con la API Local

- **Síntoma:** La app móvil se queda cargando indefinidamente o arroja Network Error al intentar iniciar sesión o cargar la flota vehicular.
- **Solución:** El smartphone no puede resolver localhost porque busca en su propio sistema operativo. Si estás probando con la API en tu computador, ingresa la IP local de tu red Wi-Fi (ej. http://192.168.1.15:8080) en lugar de localhost y confirma que el firewall de Windows permita tráfico entrante en el puerto 8080.

10.4 Fallo de Validación Hibernate (SchemaManagementException)

- **Síntoma:** El Backend aborta su arranque indicando que una tabla o columna requerida no existe.
- **Solución:** Spring Boot está configurado en modo estricto hibernate.ddl-auto: validate. Esto ocurre si no se ejecutaron las migraciones de Liquibase. Asegúrate de ejecutar el contenedor de base de datos con sus changelogs antes de iniciar el backend.